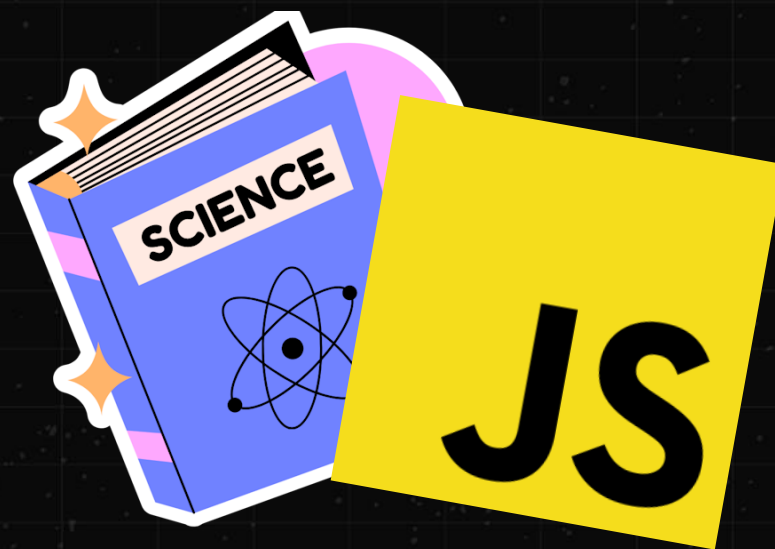


ЛЕКЦИЯ #14

CS BO FRONTEND

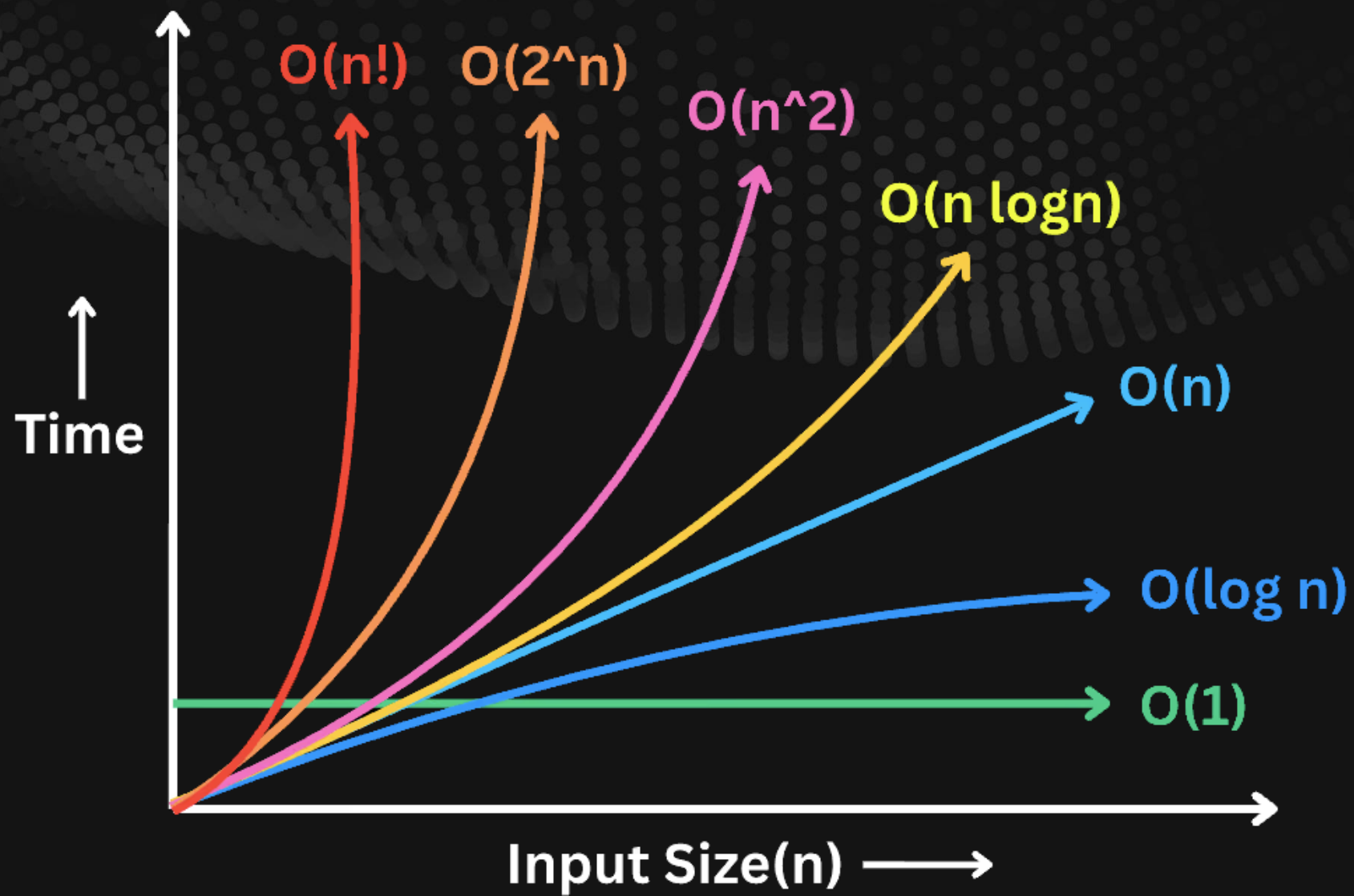


ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Вспомнить про **алгоритмическую сложность**
- * Обсудить **алгоритмы сортировки** элементов массива и что это дает
- * Познакомиться с важнейшим алгоритмом – **бинарным поиском**

АЛГ. СЛОЖНОСТЬ ПОКАЗЫВАЕТ

- * Зависимость времени выполнения алгоритма от входных данных
- * Или зависимость потребления памяти алгоритмом
- * Для оценки зависимости берутся **средние или наихудшие сценарии**
- * Например, поиск элемента в массиве в худшем случае потребует пройти все элементы массива
- * Такая сложность называется **линейной** и обозначается $O(n)$



ЗАЧЕМ ЭТО НУЖНО

- * У нас появляется **примерное понимание о требуемых ресурсах** для той или иной задачи без необходимости реальных бенчмарков
- * Например, доступ к любому элементу массива по его индексу равен $O(1)$
- * Это означает, что **неважно, сколько элементов в массиве**, ведь доступ по индексу всегда будет занимать одинаковое время
- * При анализе структур данных мы **оцениваем сложность по времени и памяти** для всех базовых операций

ПОМНИМ ПРО КЭШ И ДРУГИЕ СВОЙСТВА

- * В типовых задачах мы создаём **алгоритмы для работы в RAM**
- * Важным свойством здесь является **локальность данных** – бывает, что алгоритм с худшей сложностью, но лучшей локальностью на практике оказывается быстрее
- * Ещё один важный момент – **поддержка алгоритмом параллельных вычислений**
- * Если **алгоритм работает в других условиях**, например, для данных на HDD или в распределённой сети, то лидер может поменяться

ЧТО ЕЩЕ
ЗА **БАЗОВЫЕ**
ОПЕРАЦИИ?



У НАС ЕСТЬ КОЛЛЕКЦИИ

- * Это структуры данных для хранения других данных некоторого типа
- * Например, массивы, ассоциативные массивы, матрицы и связные списки
- * Мы можем **искать элементы в коллекции** (по индексу, ключу или признаку)
- * **Добавлять, удалять и менять местами** (если поддерживается порядок)
- * **Обходить добавленные элементы** в цикле
- * **Сравнивать коллекции** с другими такими же или создавать новые на их основе

А ВОТ КОРТЕЖ/СТРУКТУРА НЕ КОЛЛЕКЦИЯ

- * Мы не можем **добавить новые данные** в структуру — только то, что явно описано в схеме
- * Мы не можем **удалить элементы** или **обойти** их в цикле
- * Мы не можем **получить доступ к полю иначе**, кроме как по явному обращению по имени или индексу
- * Вывод: **коллекции поддерживают «изменяемость»** своего размера, а структура жёстко привязана к схеме

СВЯЗНЫЙ СПИСОК

- * Добавление / удаление / чтение **с начала** за $O(1)$
- * Добавление / удаление / чтение **с конца** за $O(n)$ или $O(1)$,
если список двусторонний и двунаправленный
- * Поиск элемента за $O(n)$
- * Элементы **следуют друг за другом** (через ссылки)
- * Чудесное свойство связного списка —
расширение памяти без копирования

ХЭШ-ТАБЛИЦА

- * Добавление / удаление / чтение **по ключу** за $O(1)$ (амортизированно)
- * Любая **отдельная операция может быть дорогой**, но последовательность из K операций выполняется за $O(K)$
- * **Поиск по предикату** (условию) за $O(n)$
- * Самая популярная **реализация ассоциативного массива**

МАССИВ

- * Чтение / запись **по индексу** за $O(1)$
- * Добавление / удаление **с конца** за $O(1)$
- * Добавление и удаление **из других мест** за $O(n)$
- * Поиск **по предикату** (условию) за $O(n)$
- * Элементы **следуют друг за другом** (рядом в памяти)

УДАЛЕНИЕ ИЗ МАССИВА ЗА $O(1)$

- * Если при удалении **поменять местами последний элемент и удаляемый**, то можно получить удаление за $O(1)$
- * Но при таком подходе **меняется порядок следования элементов**

```
function fastDelete(arr, index) {  
  if (index < 0 || index >= arr.length) {  
    throw new Error("Индекс вне границ массива");  
  }  
  
  // Меняем местами удаляемый элемент с последним  
  [arr[index], arr[arr.length - 1]] = [arr[arr.length - 1], arr[index]];  
  
  // Удаляем последний элемент (раньше он был на позиции index)  
  arr.pop();  
}  
  
const arr = [10, 20, 30, 40, 50, 60, 70];  
  
// [10, 20, 30, 40, 50, 60, 70]  
console.log(arr);  
  
// Удаляем элемент с индексом 2 (значение 30)  
fastDelete(arr, 2);  
  
// [10, 20, 70, 40, 50, 60]  
console.log(arr);
```

А МОЖНО ЛИ
УЛУЧШИТЬ **ДРУГИЕ**
ОПЕРАЦИИ В
МАССИВЕ?



ДА, НО С НЕКОТОРЫМИ ОГРАНИЧЕНИЯМИ

- * Если массив **отсортирован по значению**, по которому мы делаем поиск, то можно улучшить время поиска с $O(n)$ до $O(\log n)$
- * Причём **такая сложность сохраняется во всех случаях** и имеет математическое доказательство
- * Алгоритм такого поиска получил название «**бинарный поиск**»

```
function binarySearch(arr, target) {
  let left = 0;
  let right = arr.length - 1;

  while (left <= right) {
    const mid = left + Math.floor((right - left) / 2);

    if (arr[mid] === target) {
      return mid;
    }

    if (arr[mid] < target) {
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return -1;
}

const sorted = [10, 20, 30, 40, 50, 60, 70, 80, 90];

console.log(binarySearch(sorted, 30)); // 2
```


second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

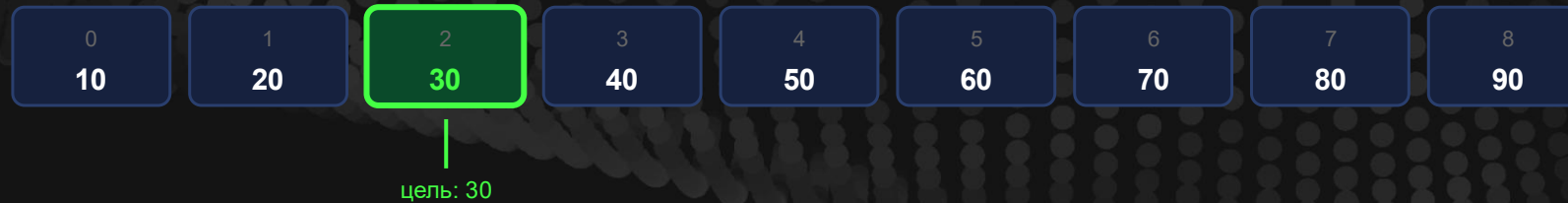
СТОЖКА, СТОЖКААА!

БЕЗ ПАНИКИ

- * Алгоритм бинарного поиска чрезвычайно прост:
берём середину массива и сравниваем её с искомым элементом
- * Если элемент меньше искомого, то повторяем алгоритм,
причём началом уже будет середина + 1
- * А если больше – концом становится середина – 1
- * Таким образом мы «делим» массив пополам на каждой итерации

Бинарный поиск (Binary Search)

Поиск элемента 30 в отсортированном массиве
Исходный массив (отсортирован по возрастанию)

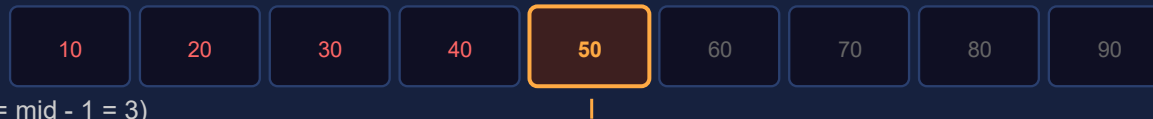


Пошаговый процесс поиска

Шаг 1

$\text{left} = 0, \text{right} = 8 \rightarrow \text{mid} = (0 + 8) / 2 = 4$

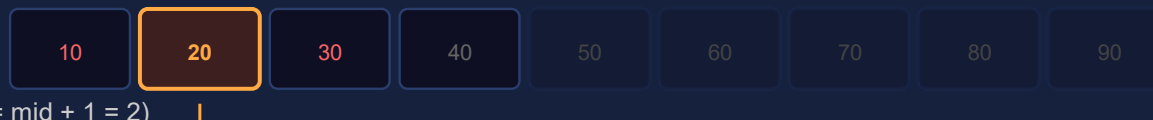
$\text{arr}[4] = 50 \rightarrow 50 > 30 \rightarrow$ ищем в левой половине ($\text{right} = \text{mid} - 1 = 3$)



Шаг 2

$\text{left} = 0, \text{right} = 3 \rightarrow \text{mid} = (0 + 3) / 2 = 1$

$\text{arr}[1] = 20 \rightarrow 20 < 30 \rightarrow$ ищем в правой половине ($\text{left} = \text{mid} + 1 = 2$)



Шаг 3 (результат)

$\text{left} = 2, \text{right} = 3 \rightarrow \text{mid} = (2 + 3) / 2 = 2 \rightarrow \text{arr}[2] = 30 \rightarrow$ элемент найден на индексе 2



Сложность бинарного поиска: $O(\log_2 n)$ — на массиве из 9 элементов потребовалось $\log_2 9 \approx 4$ проверки, нашли за 3 шага

ОПРЕДЕЛЕНИЕ СЕРЕДИНЫ

- * Чтобы найти «средний» элемент массива, можно использовать разные формулы
- * Более надёжной будет «длинная» формула: `left + Math.floor((right - left) / 2)`
- * Но есть и «короткий» вариант: `Math.floor((right + left) / 2)`
- * Меньше операций, но такая формула может привести к переполнению
- * Это особенно важно, когда мы работаем с 32-битными и меньшими числами
- * В JS работа с такими числами происходит при побитовых операциях или с типизированными массивами

УДИВИТЕЛЬНО, НО ЭТОТ АЛГОРИТМ ВЕЗДЕСУЩ

- ✱ Найти строчку, из-за которой сломался код
- ✱ Найти коммит, из-за которого сломалось приложение (в git даже команда есть)
- ✱ Найти нужную часть в книге, если её нет в оглавлении

Бинарный поиск в жизни: Git bisect

Поиск коммита, в котором сломалось приложение (git bisect)

Проблема: приложение работало, а сейчас упало. Нужно найти коммит, который всё сломал

Вручную проверять 1000 коммитов → долго. Бинарный поиск → $\log_2(1000) \approx 10$ проверок

История коммитов (от старого к новому, слева направо)



Git bisect — бинарный поиск по коммитам

Шаг 1: выбираем средний коммит между хорошим и плохим

Хороший коммит: a1b2c3d (работает) | Плохой коммит: x9y8z7w (сломано)

Средний коммит: f4e5d6c → проверяем, работает ли приложение

f4e5d6c
проверяем → ?

Шаг 2: смотрим результат проверки

Если коммит f4e5d6c работает → плохой коммит правее (в более новых коммитах)

Если коммит f4e5d6c сломан → плохой коммит левее (старее)

✗ сломан → ищем левее

✓ работает → ищем правее

Шаг 3: повторяем алгоритм

Снова выбираем средний коммит в новом диапазоне, проверяем, сужаем область поиска

Шаг 4 (результат): найден коммит, где приложение сломалось

За $\log_2(n)$ проверок находим виновный коммит — можно чинить!

```
git bisect start → git bisect good a1b2c3d → git bisect bad x9y8z7w → git проверяет → git bisect reset
```

* Вместо ручной проверки 1000 коммитов ($O(n)$) — бинарный поиск находит виновный за $O(\log_2 n) \approx 10$ проверок

НО КАК **БЫТЬ**
СО ВСТАВКОЙ
И УДАЛЕНИЕМ?



МОЖНО УДАЛИТЬ ИЛИ ВСТАВИТЬ ЗА $O(1)$

- * А потом отсортировать массив
- * Проблема в том, что это достаточно дорогая операция
- * Сложность сортировки зависит как от самих данных, так и от выбранного алгоритма
- * Наивные алгоритмы работают за квадратичное время $O(n^2)$
- * Хорошие работают за $O(n \log n)$, но это всё равно немало
- * Некоторые могут работать за $O(n)$, но привязаны к типам данных

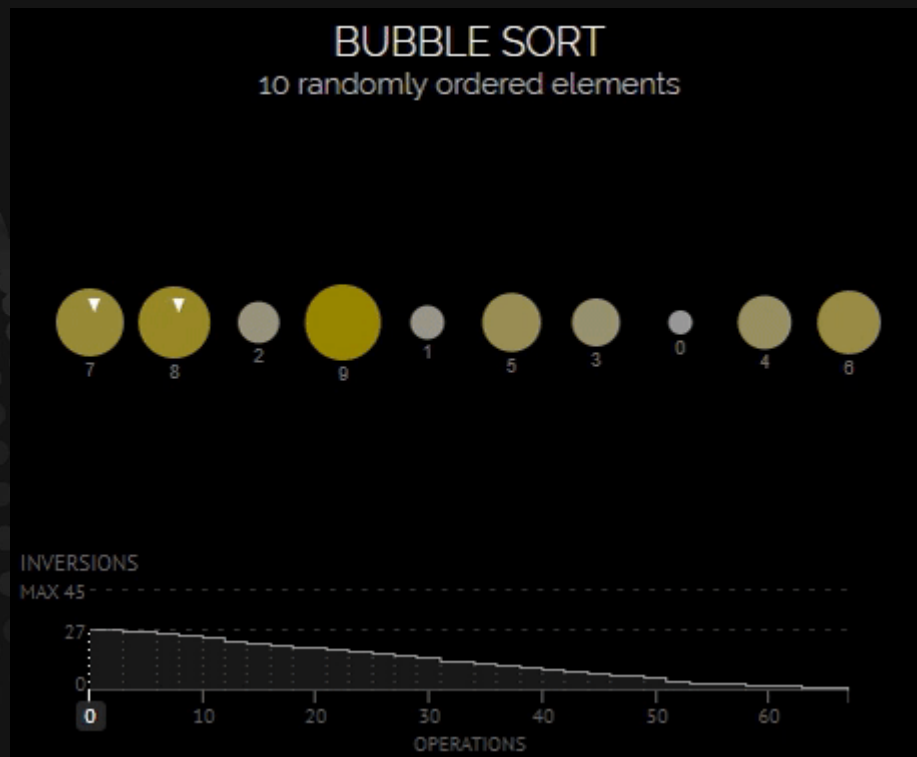
ЭТО ЦЕЛАЯ НАУКА

- * Алгоритмов сортировки несчетное множество
- * Разница в алгоритмической сложности **по времени и памяти**
- * К тому же некоторые алгоритмы **могут деградировать до более плохой сложности**
- * В **устойчивости** сортировки (сортировка одинаковых ключей)
- * В применимости к **конкретным типам данных** (только числа)
- * И к **конкретным видам памяти**: RAM, SSD, HDD и так далее
- * В возможности **распараллеливать вычисления**

ПОПУЛЯРНЫЕ АЛГОРИТМЫ

- * Пузырьковая сортировка – $O(n^2)$ в среднем, $O(1)$ памяти, простая, но медленная
- * Быстрая сортировка – $O(n \log n)$ в среднем, но деградирует до $O(n^2)$, нестабильная
- * Сортировка слиянием – стабильные $O(n \log n)$ во всех случаях, но требует $O(n)$ дополнительной памяти
- * Сортировка кучей – гарантированные $O(n \log n)$ и $O(1)$ памяти, но нестабильная
- * Поразрядная сортировка – линейная $O(n \cdot k)$, но привязана к типу данных (числа или строки)

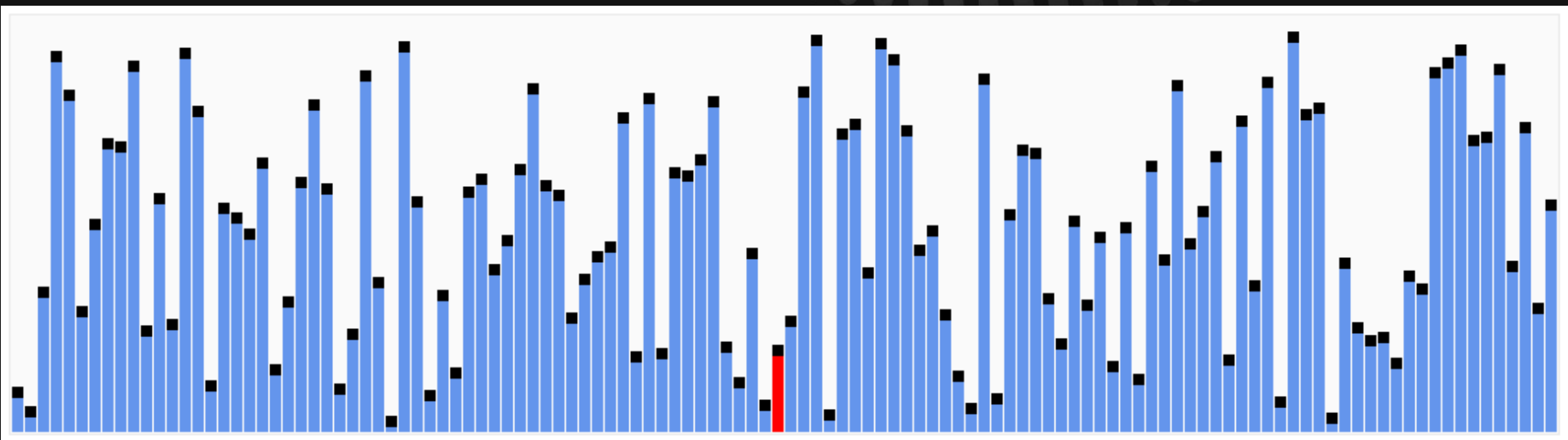
ПУЗЫРЬКОВАЯ СОРТИРОВКА (BUBBLESORT)



БЫСТРАЯ СОРТИРОВКА (QUICKSORT)

- * Обменная сортировка (divide and conquer), **неустойчивая сортировка**
- * **Одна из самых быстрых сортировок** для RAM на практике
- * Требуется $O(\log n)$ памяти на стеке вызовов
- * В среднем работает за $O(n \log n)$, но деградирует до $O(n^2)$
- * На почти отсортированных данных может деградировать без хорошей стратегии выбора опорного элемента
- * Подходит как для больших, так и для малых объёмов
- * **Эффективно использует кэш** и может распараллеливаться

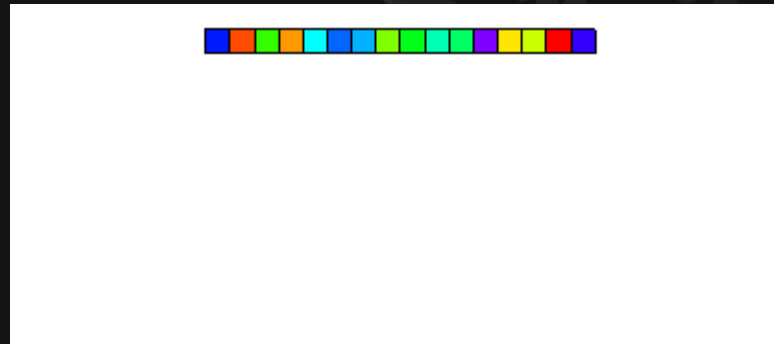
ВИЗУАЛИЗИРУЕМ



СОРТИРОВКА СЛИЯНИЕМ (MERGESORT)

- * Стабильная сортировка
- * Отлично подходит для памяти с последовательным доступом, например HDD
- * Стабильная сложность по времени $O(n \log n)$ и сложность по памяти $O(n)$
- * Для почти отсортированных массивов работает так же хорошо
- * Для малых объёмов не очень эффективна
- * Эффективно использует кэш и может распараллеливаться

ВИЗУАЛИЗИРУЕМ



СОРТИРОВКА КУЧЕЙ (HEAPSORT)

- * Построена на использовании **сбалансированного дерева «Бинарная куча»**
- * **Нестабильная сортировка**
- * Эффективна в RAM и не требует дополнительной памяти
- * **Гарантированная сложность** по времени $O(n \log n)$
- * Для почти отсортированных массивов работает так же
- * **Плохо поддерживает кэш** и не распараллеливается

ВИЗУАЛИЗИРУЕМ

10	4	8	5	12	2	6	11	3	9	7	1
----	---	---	---	----	---	---	----	---	---	---	---

ПОРАЗРЯДНАЯ СОРТИРОВКА (RADIXSORT)

- * Работает только для целых чисел и строк фиксированной длины
- * Эффективна в RAM
- * Работает за линейное время, но требует дополнительной памяти

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВЫДЫХАЕМ

- * Идеального алгоритма не существует
- * Нужно **составить в голове хотя бы примерную карту**, где что лучше использовать
- * **Помним про стабильность** (устойчивость)
- * В 90% случаев хватит стандартной реализации
- * Но иногда использование другого алгоритма **может дать драматический эффект**
- * Например, RadixSort может быть быстрее в десятки раз

ЧТО ЕЩЕ ЗА
СТАБИЛЬНОСТЬ?





```
const students = [  
  { group: 2, name: "Анна" },  
  { group: 1, name: "Иван" },  
  { group: 2, name: "Борис" },  
  { group: 1, name: "Пётр" }  
];  
  
students.sort((a,b) => a.group - b.group);  
  
// [  
//   { group: 1, name: 'Иван' },  
//   { group: 1, name: 'Пётр' },  
//   { group: 2, name: 'Анна' },  
//   { group: 2, name: 'Борис' }  
// ];  
console.log(students);
```

В ЧЕМ ТУТ СОЛЬ

- * В стабильной (устойчивой) сортировке мы точно знаем, что значения с одинаковым ключом сортировки сохраняют свой естественный порядок
- * Например, { group: 2, name: "Борис" } в нашем примере всегда будет после { group: 2, name: "Анна" }
- * А нестабильная сортировка такой гарантии не даёт
- * Проблема в том, что нестабильная сортировка на практике работает быстрее

СОРТИРОВКА И JS

- * В JS у массивов есть встроенные методы `sort` (сортирует текущий) и `toSorted` (создает новый отсортированный)
- * По умолчанию значения сортируются как строки (лексикографически), но можно передать функцию-компаратор
- * Начиная со стандарта ECMAScript 2019, спецификация явно требует **использовать алгоритм, дающий стабильную сортировку**
- * В V8 раньше использовался QuickSort, а теперь TimSort (гибридный алгоритм, использующий под капотом сортировку вставками и слиянием)

НАПРИМЕР

```
const arr = [1, 2, 3, 10, 0];

// [ 0, 1, 10, 2, 3 ]
console.log(arr.sort());

// [ 0, 1, 2, 3, 10 ]
console.log(arr.sort((a, b) => a - b));

// [ 10, 3, 2, 1, 0 ]
console.log(arr.sort((a, b) => b - a));
```


ПОРЯДОК В АССОЦИАТИВНЫХ МАССИВАХ

- ✱ В JS ассоциативные массивы сохраняют порядок вставки (у объектов для ключей-чисел это не так), поэтому можно реализовать сортировку и для них
- ✱ Но на практике такое нужно не очень часто – проще **создать отдельный массив**

ТАК ЧТО ТАМ
СО **ВСТАВКОЙ**
И УДАЛЕНИЕМ?



ДА, МЫ ОТВЛЕКЛИСЬ

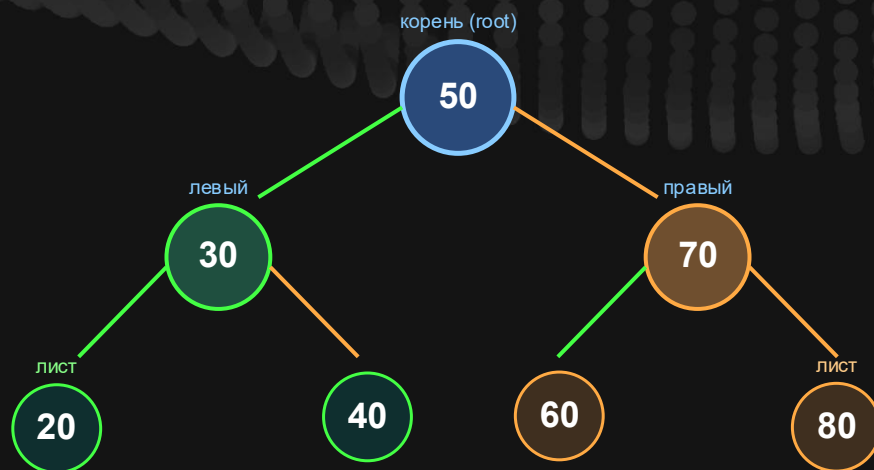
- * Проблема массивов в том, что даже если мы быстро найдём место для вставки или удаления, нам **придётся двигать элементы в массиве**
- * Либо **вставлять в конец и делать сортировку**
- * Однако **решить данную проблему можно**, используя подход связного списка

Бинарное дерево поиска (Binary Search Tree)

Каждый левый потомок меньше родителя, каждый правый — больше

левый < родитель

правый > родитель



Для любого узла: все значения в левом поддереве меньше узла, все значения в правом — больше

Поиск · Вставка · Удаление → $O(\log n)$ в среднем ($O(n)$ в худшем, если дерево несбалансировано)

РЕФЛЕКСИРУЕМ

- * Используя сортировку и связный список, можно сделать структуру, которая **работает за логарифмическое время на любых операциях изменения**
- * А также **предоставляет дополнительные свойства**
- * Но фокус в том, что для эффективности дерева **все его ветви должны иметь одинаковую высоту**
- * Такое дерево **называется сбалансированным**
- * У нас будет несколько лекций по этой теме

А КАК ЖЕ
ЛОКАЛЬНОСТЬ?



ДА, ЭТО ПРОБЛЕМА

- * Связный список имеет плохую локальность, **но его можно улучшить**, сделав связный список массивов
- * Такой же подход работает и с деревьями: существуют структуры, где **в узле находится массив значений**
- * Такие деревья **называются многопутевыми**, а не двоичными
- * Например, R-дерево или B-дерево реализуются таким образом
- * Но на самом деле **даже бинарное дерево можно представить массивом**
- * Мы разберём эти способы отдельно

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

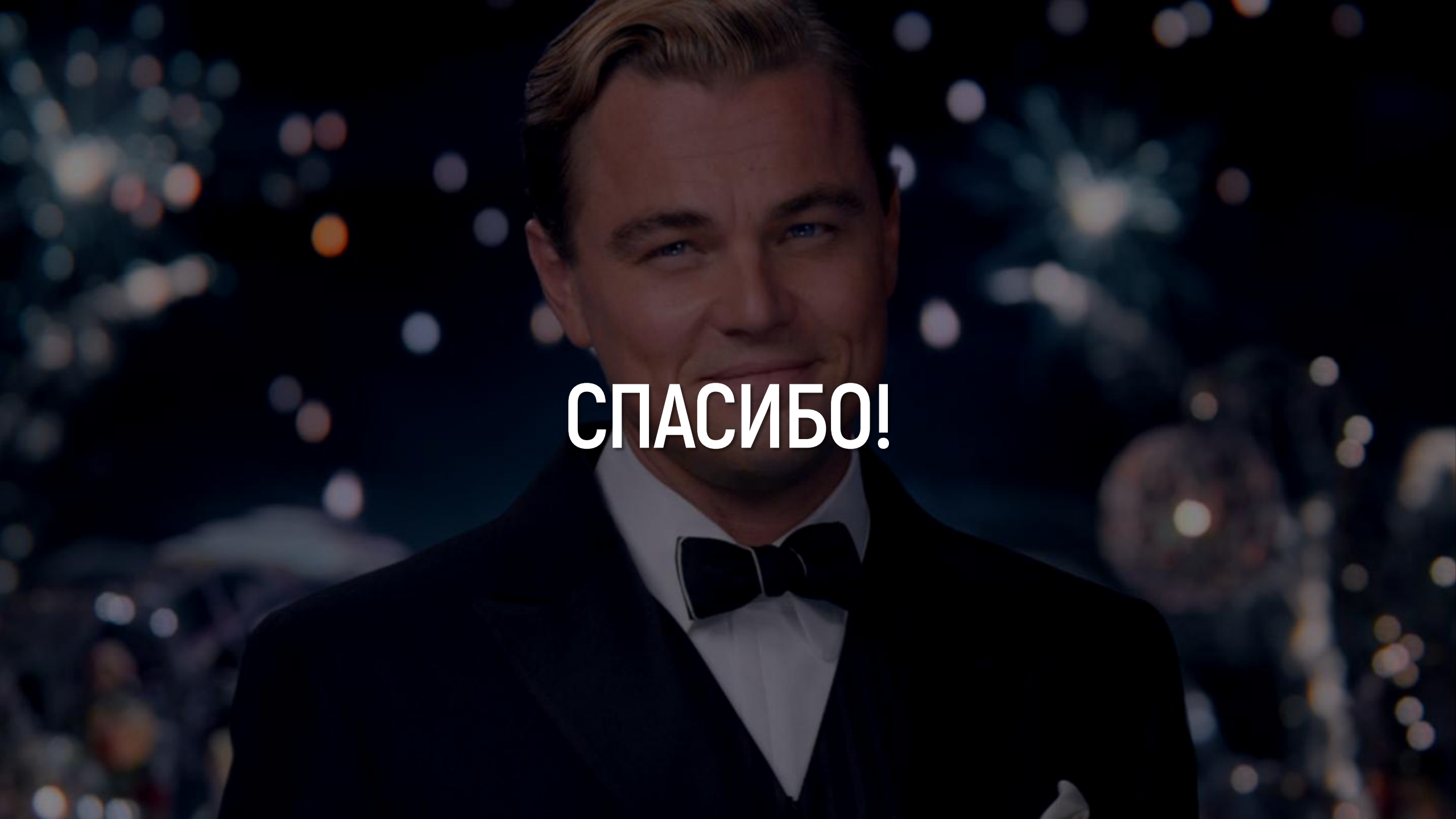
СТОЖКА, СТОЖКААА!

МЫ ЗАБЕГАЕМ ВПЕРЕД

- * Использование идеи бинарного поиска **лежит в основе деревьев поиска**
- * Только они **улучшают операции вставки и удаления**, чтобы они также работали за логарифмическое время
- * Деревья поиска могут использоваться и **для реализации ассоциативных массивов**
- * На практике оно может быть медленнее хэш-таблицы, но **имеет другие сильные стороны** (например, поиск экстремумов)
- * Но для этого **дерево должно быть сбалансированным**
- * Поговорим про это в другой раз

ПОДВЕДЕМ ИТОГИ

- * Порядок данных в коллекции имеет большие последствия
- * Если данные отсортированы по некоторому ключу, то мы можем очень эффективно искать по нему, **используя алгоритм бинарного поиска**
- * Сортировка — это всегда **компромисс** между скоростью, памятью и стабильностью
- * На практике важна не только O -нотация, но и **локальность данных** и параллелизм
- * Сложные структуры **строятся из простых**

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus bokeh lights in various colors (white, yellow, orange, blue), suggesting a night-time event or fireworks.

СПАСИБО!